

Deep Learning para visión

Informe técnico del proyecto final

Detección y localización de daños visibles en carrocería de vehículos mediante modelos de detección de objetos basados en YOLO

Anastasie Wagenheim

Abril 2026

Índice

1. Introducción	3
2. Revisión del estado del arte	3
3. Descripción del dataset	5
3.1 Origen de los datos	5
3.2 División del dataset	5
3.3 Clases	6
3.4 Anotaciones	6
3.5 Resolución y preprocesamiento previo	7
3.6 Justificación de su uso	7
3.7 Uso del dataset COCO para evaluación de falsos positivos	7
4. Procesamiento de datos	8
4.1 Preprocesamiento	8
4.2 Data augmentation	8
5. Arquitectura y diseño del modelo	9
5.1 Arquitectura baseline seleccionada: YOLOv8	9
5.2 Variante del modelo	9
5.3 Arquitectura interna e hiperparámetros	10
6. Entrenamiento y evaluación	10
6.1 Diseño experimental	10
6.2 Métricas	11
6.3 Evaluación cualitativa	12
6.4 Evaluación de falsos positivos	12
6.5 Resultados cuantitativos	12
7. Análisis crítico	16
7.1 Limitaciones del enfoque y problemas encontrados	16
7.2 Posibles mejoras	16
8. Conclusiones	17
9. Referencias	18

1. Introducción

En los últimos años, el avance del Deep Learning ha impulsado significativamente el desarrollo de sistemas automatizados de visión en múltiples dominios. Entre ellos, la detección de objetos en imágenes se ha consolidado como una de las tareas principales de computer vision, con aplicaciones dentro de la conducción autónoma, la inspección industrial o la vigilancia.

Este proyecto se centra en la detección y localización de daños visibles en la carrocería de vehículos, especialmente relevante en los sectores de la automoción o de los seguros. Actualmente, la identificación de daños en vehículos se hace mayormente de forma manual, lo que implica unos procesos costosos, así como la subjetividad inherente al criterio humano. El desarrollo de sistemas automatizados que permitan realizar esta tarea de manera más objetiva y eficiente resulta un proyecto a la orden del día y con fines aplicables al mundo real.

El objetivo principal de este proyecto es diseñar e implementar una solución basada en modelos de detección del grupo YOLO (You Only Look Once), una arquitectura de redes neuronales especializada en detección de objetos en tiempo real, combinando precisión y eficiencia. Con este sistema, se podrán identificar distintos tipos de daños en vehículos, localizando su posición a través de bounding boxes.

Para abordar el problema de forma experimental y rigurosa, se plantea un enfoque comparativo basado en distintas configuraciones del modelo YOLO. En primer lugar, se establecerá un modelo baseline como punto de referencia. A partir de este, se realiza una comparación entre diferentes variantes de la arquitectura, para analizar el compromiso entre velocidad y precisión. Posteriormente, se estudiará el impacto de técnicas de data augmentation y fine-tuning sobre el rendimiento del modelo, evaluando su capacidad de generalización.

Para ello, utilizaremos un dataset específico del dominio de daños en vehículos, lo que permitirá entrenar el modelo en un entorno realista y representativo del problema, que se formula como una tarea de detección de objetos multi-clase basada en redes neuronales convolucionales profundas.

2. Revisión del estado del arte

La detección de daños en la carrocería de vehículos forma parte del campo de computer vision aplicada a la inspección industrial que, gracias al desarrollo del Deep Learning, ha experimentado un crecimiento significativo. Tradicionalmente, la inspección de daños se solía realizar manualmente, con procesos humanos costosos y una gran posibilidad de introducir un sesgo subjetivo en el análisis. Con el objetivo de automatizar estas tareas, las primeras pruebas de computer vision se basaron en técnicas clásicas de procesamiento de imágenes, como la detección de bordes o

segmentación con umbrales. Estos métodos, si bien automatizables y escalables, presentaban poca robustez frente a cambios de iluminación, ángulos o diversidad de los daños.

Con el avance de las redes neuronales convolucionales (CNN), la detección de objetos cambió radicalmente: se empezaron a utilizar regiones de interés para localizar objetos, con alta precisión, con modelos como R-CNN, Fast y Faster R-CNN. En concreto, las CNN fueron revolucionarias en este ámbito porque permitieron eliminar la necesidad de un diseño manual de las características de una imagen. Con las CNN, el modelo aprende directamente de los datos.

Más adelante aparecieron modelos one-stage como SSD (Single Shot MultiBox Detector) o YOLO (You Only Look Once), que constituyeron un compromiso sólido entre velocidad y precisión. Los detectores anteriores como Faster R-CNN funcionaban en dos pasos diferenciados: proponían regiones candidatas y clasificaban luego cada región, además de ajustar las bounding boxes. Los modelos one-stage combinan estas dos etapas en una sola, dividiendo la imagen en un grid, a partir del cual predicen directamente los bounding boxes, la clase del objeto y su probabilidad.

En particular, YOLO evolucionó en múltiples versiones (YOLOv3, YOLOv5, YOLOv8), consolidándose como una de las arquitecturas más utilizadas en problemas de detección de imágenes en tiempo real.

En el ámbito concreto de la detección de daños en vehículos, la literatura reciente ha explorado principalmente el uso de estas arquitecturas adaptadas a datasets especializados. Los enfoques más relevantes incluyen:

- Aplicación de modelos YOLO para la detección directa de daños como arañazos, abolladuras o grietas mediante bounding boxes.
- Uso de transfer learning a partir de modelos preentrenados en datasets generales como COCO, adaptándolos a dominios específicos del sector automotriz.
- Combinación de detección de objetos y segmentación semántica para mejorar la localización de daños pequeños o de contornos poco definidos.
- Estrategias de data augmentation para mejorar la robustez del modelo ante variaciones de iluminación, resolución y perspectiva.

En el sector industrial, estas soluciones empiezan a integrarse en sistemas de inspección automática de vehículos, tanto en empresas de automoción como de seguros. Sin embargo, a pesar de los avances en arquitecturas de detección, los modelos presentan limitaciones en la detección de objetos pequeños y en escenarios con alta variabilidad visual, como es el caso de los daños superficiales en vehículos.

En este proyecto se opta en el baseline por una arquitectura one-stage como YOLO debido a la necesidad de un equilibrio entre precisión y eficiencia computacional, especialmente relevante en escenarios de aplicación real donde la inferencia en tiempo real es un factor crítico.

3. Descripción del dataset

3.1 Origen de los datos

Para este proyecto, utilizaremos el dataset [Car damage V5](#), disponible en la plataforma Roboflow. Este conjunto de datos está diseñado específicamente para tareas de detección de objetos (object detection) en el dominio de la automoción, centrado en la identificación de distintos tipos de daños visibles en vehículos. Contiene imágenes de vehículos en distintas condiciones, presentando daños visibles en la carrocería, de gravedad variable. Se incluyen distintos ángulos de captura, condiciones de iluminación y fondos o entornos.

El dataset ha sido recopilado y etiquetado dentro de la comunidad de Roboflow, una plataforma que proporciona datasets preparados para tareas de computer vision. En su versión original, las anotaciones se proporcionan como polígonos de segmentación, que delimitan precisamente las regiones dañadas del vehículo. En este proyecto, dichas anotaciones se transforman a formato de bounding boxes, con el objetivo de adaptarlas a modelos de detección de objetos como YOLO, simplificando el problema a la localización de regiones rectangulares que contienen el daño.

3.2 División del dataset

El dataset se divide en 3 subconjuntos estándar:

Conjunto	Número de imágenes
Train	6085
Validation	572
Test	320
Total	6977

Esta partición sigue el esquema habitual en problemas de aprendizaje supervisado; permite separar el proceso de entrenamiento, ajuste de hiperparámetros y evaluación final del modelo, sobre datos no vistos.

3.3 Clases

El dataset se compone de 4 clases que representan daños en vehículos, de distinta gravedad:

Clase	Descripción	Número de instancias
0	Broken glass (cristal roto, incluyendo luces y faros)	1619
1	Dent (abolladura)	2027
2	Scratch (rasguños)	4138
3	Wreck (daño estructural)	2652

Se observa una predominancia de la clase Scratch, alineado con la naturaleza del problema, ya que los arañazos representan el tipo de daño más frecuente en carrocería de vehículos.

Este desbalance entre clases se considera moderado, ya que cada una cuenta con un número representativo de ejemplos, lo que permite el entrenamiento de modelos supervisados sin tener que aplicar técnicas de balanceo. Además, esta repartición refleja bastante bien la distribución de los daños en el mundo real, aportando realismo al problema.

Finalmente, se mantienen las clases definidas originalmente en el dataset porque no presentan solapamiento semántico grave, permitiendo un análisis detallado del modelo mediante una evaluación comparativa por tipo de daño.

3.4 Anotaciones

El dataset original está anotado con polígonos de segmentación, que permiten localizar espacialmente cada daño dentro de la imagen, delimitando con precisión la región afectada.

Para este proyecto, estas anotaciones han sido transformadas en bounding boxes, siguiendo el estándar utilizado en tareas de detección de objetos. Este formato es compatible con modelos de YOLO e incluye la clase del objeto y las coordenadas normalizadas del rectángulo delimitador (x_center, y_center, width, height).

Esta transformación simplifica el problema original, reduciendo el polígono a una región rectangular. Si bien esto implica una pérdida de precisión en la forma, permite una mayor eficiencia computacional y facilita el entrenamiento de modelos de detección en tiempo real.

3.5 Resolución y preprocesamiento previo

El dataset de Roboflow ha sido previamente preprocesado, concretamente afectando la resolución de las imágenes. En efecto, han sido estandarizadas a una resolución fija de 640×640 píxeles, mediante una operación de stretching, adaptando directamente la imagen a la resolución definida, sin siempre preservar la relación de aspecto.

Este preprocesamiento asegura homogeneidad en la entrada del modelo y facilita el entrenamiento de redes neuronales convolucionales como YOLOv8. Sin embargo, el uso de stretch puede introducir ligeras distorsiones geométricas, lo cual podría afectar a la localización precisa de daños pequeños.

3.6 Justificación de su uso

Este dataset está diseñado específicamente para detectar daños en vehículos, lo que permite abordar el proyecto de forma directa, evitando la necesidad de adaptar datasets genéricos no representativos del dominio. También nos permite centrar el análisis en el rendimiento del modelo, en lugar de en tareas de adaptación o reinterpretación de los datos.

Además, el dataset incluye anotaciones que se basan en regiones que delimitan los daños en los vehículos. En nuestro caso, se definieron como contornos precisos (polígonos), que se han transformado a bounding boxes para poder utilizarse en modelos de detección de objetos tipo YOLO. Esta transformación permite simplificar el problema a una tarea de detección estándar, manteniendo la información espacial más relevante del daño.

Esta decisión se justifica por el objetivo del proyecto, centrado en detección en tiempo real, donde el uso de bounding boxes resulta más eficiente que la segmentación a nivel de píxel.

Asimismo, el dataset incluye variabilidad visual en las imágenes, lo que contribuye a mejorar la capacidad de generalización del modelo. Con distintas iluminaciones, ángulos y tipos de daño, se aproxima a un escenario real.

Finalmente, el uso de este dataset resulta coherente con el objetivo del proyecto, facilitando la evaluación comparativa entre distintos modelos y configuraciones, y eso en un entorno controlado y reproducible, lo que es clave para un análisis experimental.

3.7 Uso del dataset COCO para evaluación de falsos positivos

Para complementar la evaluación con las métricas estándares, construimos un conjunto adicional de imágenes sin presencia de daños visibles (no damage), para poder analizar los falsos positivos del modelo.

Para ello, utilizaremos el dataset COCO 2017, del cual seleccionamos un subconjunto de imágenes de la clase “car”. A partir de este conjunto, extraemos 100 imágenes sin anotaciones de daños, para ser utilizadas como casos negativos en la evaluación.

Esta estrategia permite analizar el comportamiento del modelo en escenarios realistas donde no existe ningún tipo de daño, evaluando su capacidad para no generar detecciones falsas. Es importante destacar que estas imágenes no forman parte del conjunto de entrenamiento ni validación, y se utilizan únicamente con fines de evaluación externa del modelo.

4. Procesamiento de datos

4.1 Preprocesamiento

Antes del entrenamiento, hacemos un análisis exploratorio del dataset (sanity check), para revisar que tanto las imágenes como las anotaciones y las clases estén listas para el modelo. Confirmamos en esta etapa que el desbalance entre clases es moderado y decidimos no aplicar técnicas de balanceo para trabajar con un conjunto fiel a la representación del mundo real.

Además, la resolución de imágenes se estandarizó previamente, por lo que no es necesario redimensionarlas nuevamente.

4.2 Data augmentation

El framework seleccionado, YOLO, permite incorporar de forma nativa un conjunto de técnicas de data augmentation aplicadas dinámicamente durante el entrenamiento. Se pueden incluir:

- transformaciones geométricas:
 - flipping horizontal
 - scaling (cambio de escala)
 - translation (desplazamiento)
- variaciones fotométricas en el espacio HSV: alteraciones en tono, saturación y brillo, que modifican la apariencia de la imagen sin alterar la geometría.
- técnicas de composición:
 - mosaic, que combina cuatro imágenes en una sola, favoreciendo la detección de objetos en distintos contextos y escalas
 - mixup, que genera nuevas muestras mediante la combinación de dos imágenes y sus respectivas anotaciones

Estas transformaciones se aplican de forma automática durante el entrenamiento del modelo, sin necesidad de generar un dataset con nuevas imágenes de manera explícita.

En este proyecto, se consideran especialmente potentes las técnicas de mosaic, flipping horizontal, variaciones en HSV, scaling y translation, ya que permiten simular condiciones habituales en escenarios reales de conducción, como cambios de iluminación, variaciones de perspectiva, diferencias de escala y oclusiones parciales de

los vehículos. Esto contribuye a mejorar la capacidad de generalización del modelo frente a datos no vistos.

Para el baseline, incluimos estas transformaciones más relevantes, mientras que no activamos técnicas como mixup o shear, manteniéndolas con valor 0 (defecto).

Este baseline permite establecer una referencia sólida contra la que evaluaremos las modificaciones introducidas en los distintos experimentos de fine-tuning.

5. Arquitectura y diseño del modelo

5.1 Arquitectura baseline seleccionada: YOLOv8

Para este proyecto, seleccionamos la arquitectura YOLOv8 (You Only Look Once), un modelo de detección de objetos basado en redes neuronales convolucionales (CNN), especialmente diseñado para trabajar en tiempo real.

YOLOv8 pertenece a los modelos one-stage y anchor-free, que predice directamente bounding boxes y clases en una única pasada, mientras que los modelos two-stages como por ejemplo Faster R-CNN, dividen el proceso entre generación de regiones candidatas y luego su clasificación. Esta diferencia implica un trade-off entre precisión y velocidad.

En comparación con alternativas clásicas:

- Faster R-CNN: muy preciso pero más lento, y con un coste computacional elevado.
- YOLOv5: YOLOv8 introduce mejoras en eficiencia del entrenamiento y estabilidad.

YOLOv8 se considera la opción más adecuada para este problema al ofrecer un equilibrio entre precisión, eficiencia computacional y facilidad de despliegue.

5.2 Variante del modelo

Dentro de las variantes de YOLOv8, disponemos de:

- YOLOv8n (nano): máxima eficiencia computacional, pero menor capacidad de representación, lo que puede afectar a la detección de daños pequeños o poco visibles.
- YOLOv8s (small): equilibrio entre precisión y coste computacional.
- YOLOv8m (medium): mayor capacidad de aprendizaje, pero con un incremento significativo del coste computacional.

En este proyecto se selecciona YOLOv8s como modelo baseline, ya que proporciona el mejor compromiso entre rendimiento y eficiencia para un dataset de tamaño medio como el de este proyecto.

5.3 Arquitectura interna e hiperparámetros

YOLOv8 se basa en una arquitectura CNN compuesta por tres bloques principales:

- **Backbone:** extrae las características visuales a diferentes niveles de abstracción mediante convoluciones sucesivas.
- **Neck:** combina características multiescala mediante mecanismos tipo feature pyramid, lo que permite mejorar la detección de objetos de distintos tamaños.
- **Head:** realiza la predicción final de bounding boxes, clases y probabilidades asociadas.

Las capas convolucionales utilizan funciones de activación no lineales tipo SiLU (Sigmoid Linear Unit), lo que mejora la estabilidad del entrenamiento y el rendimiento frente a activaciones clásicas como ReLU.

Los hiperparámetros principales utilizados son:

Hiperparámetro	Valor	Comentario
Epochs	50	Estándar para un baseline de computer vision, permite convergencia sin coste computacional excesivo
Image size	640	Estándar para YOLO, da un buen equilibrio entre detalles y coste computacional
Batch size	16	Compromiso entre estabilidad y velocidad

6. Entrenamiento y evaluación

6.1 Diseño experimental

Para abordar el problema con rigor, planteamos un enfoque experimental basado en la comparación de distintas configuraciones de YOLOv8.

Implementamos un modelo baseline basado en YOLOv8s con las configuraciones por defecto del framework, que sirve de referencia. A partir de este, diseñamos distintos experimentos para poder analizar el impacto de diferentes factores en el rendimiento del modelo:

Experimento	Modelo	Comentario
0: Baseline	yolov8s	Modelo de referencia.

Experimento	Modelo	Comentario
1: versión nano	yolov8n	Comparamos con una arquitectura más ligera para evaluar el trade-off entre eficiencia computacional y precisión de las predicciones.
2: augmentation tuning	yolov8s_ft	Ajustamos el data augmentation para introducir rotación, shear y mixup, técnicas candidatas para mejorar el baseline.
3: no mixup	yolov8s_ft_2	Analizamos el impacto de mixup, eliminándolo de las técnicas ajustadas previamente.
4: imgsz=768	yolov8s_ft_3	Aumentamos la resolución de las imágenes de entrada que, junto con los ajustes de data augmentation validadas previamente, podrían mejorar la detección de datos pequeños.

Todos los modelos se entrenan bajo condiciones controladas (mismo dataset, número de épocas y batch size), permitiendo una comparación justa entre ellos.

6.2 Métricas

Para evaluar el rendimiento del modelo se utilizan métricas estándar en detección de objetos, que permiten medir tanto la calidad de las predicciones como la precisión en la localización de los daños:

- Precision: mide la proporción de predicciones correctas sobre el total de predicciones realizadas.
- Recall: mide la capacidad del modelo para detectar todos los objetos presentes.
- mAP@0.5: media de la precisión promedio considerando un umbral de intersection over union (IoU) de 0.5. Evalúa si las detecciones son correctas en términos generales.
- mAP@0.5:0.95: métrica más estricta que promedia el rendimiento para un umbral de IoU entre 0.5 y 0.95. Evalúa si las bounding boxes están correctamente situadas en la imagen.

Estas métricas permiten evaluar tanto la calidad de las detecciones como la precisión en la localización de los objetos, de manera comparativa, entre las distintas configuraciones del modelo YOLOv8. Se calculan tanto a nivel global como por clase, lo que permite analizar el comportamiento del modelo en cada tipo de daño.

6.3 Evaluación cualitativa

Además de las métricas numéricas, se realiza un análisis cualitativo de las predicciones del modelo sobre imágenes del conjunto de test.

Este análisis consiste en comparar visualmente las anotaciones reales (ground truth) y las predicciones generadas por el modelo.

El objetivo es múltiple:

- Evaluar la calidad de las bounding boxes
- Analizar la confianza de las predicciones
- Identificar errores recurrentes (omisiones, duplicaciones)

Este tipo de evaluación resulta especialmente útil en problemas como este, donde pequeños detalles visuales (por ejemplo, arañazos) pueden ser difíciles de capturar.

6.4 Evaluación de falsos positivos

Introducimos finalmente un análisis específico de falsos positivos en escenarios negativos, a partir de un conjunto de imágenes sin daño seleccionadas, procedentes del dataset COCO. Incluye imágenes de vehículos sin daños visibles y se utiliza exclusivamente para evaluación (no forma parte del entrenamiento).

El objetivo también es plural:

- Evaluar la capacidad del modelo para no detectar daños inexistentes
- Medir la tasa de falsos positivos en escenarios reales
- Analizar el comportamiento del modelo en contextos visualmente complejos

Este análisis resulta clave en aplicaciones reales (por ejemplo, seguros o inspección automática), donde detectar daños inexistentes puede tener un impacto significativo.

6.5 Resultados cuantitativos

Tabla de resultados cuantitativos globales

	Modelo	mAP50	mAP50-95	Precision	Recall
0	yolov8s	0.869	0.681	0.892	0.812
1	yolov8n	0.812	0.597	0.848	0.756
2	yolov8s_ft	0.847	0.627	0.873	0.790
3	yolov8s_ft_2	0.854	0.646	0.864	0.815
4	yolov8s_ft_3	0.860	0.655	0.875	0.833

A nivel global, el modelo YOLOv8s (baseline) presenta el mejor valor de mAP@0.5 (0.869) y precision (0.892). La variante YOLOv8n muestra un descenso generalizado en todas las métricas, reflejando su menor capacidad de representación.

En cuanto a los modelos ajustados, yolov8s_ft presenta una ligera reducción en todas las métricas respecto al baseline. Por su parte, yolov8s_ft_2 mejora el recall global, aunque mantiene valores de mAP inferiores al baseline.

El modelo yolov8s_ft_3, entrenado con mayor resolución de entrada, alcanza el mayor valor de recall global (0.833) y mejora el mAP@0.5:0.95 respecto a los modelos anteriores con fine-tuning. No obstante, su precisión (0.875) sigue siendo ligeramente inferior a la del baseline.

Tablas de resultados cuantitativos por clase

Precision

	Modelo	Broken	Glass	Dent	Scratch	Wreck
0	yolov8s		0.886	0.929	0.865	0.886
1	yolov8n		0.876	0.896	0.779	0.842
2	yolov8s_ft		0.833	0.897	0.874	0.890
3	yolov8s_ft_2		0.829	0.892	0.854	0.881
4	yolov8s_ft_3		0.859	0.907	0.846	0.889

mAP50

	Modelo	Broken	Glass	Dent	Scratch	Wreck
0	yolov8s		0.890	0.910	0.766	0.908
1	yolov8n		0.878	0.835	0.662	0.873
2	yolov8s_ft		0.874	0.871	0.737	0.906
3	yolov8s_ft_2		0.872	0.886	0.763	0.896
4	yolov8s_ft_3		0.862	0.899	0.772	0.908

Recall

	Modelo	Broken	Glass	Dent	Scratch	Wreck
0	yolov8s		0.867	0.841	0.687	0.854
1	yolov8n		0.899	0.752	0.558	0.814
2	yolov8s_ft		0.867	0.814	0.629	0.849
3	yolov8s_ft_2		0.892	0.858	0.675	0.834
4	yolov8s_ft_3		0.899	0.885	0.709	0.839

mAP50-95

	Modelo	Broken	Glass	Dent	Scratch	Wreck
0	yolov8s		0.782	0.705	0.538	0.698
1	yolov8n		0.707	0.613	0.450	0.618
2	yolov8s_ft		0.718	0.652	0.498	0.639
3	yolov8s_ft_2		0.740	0.678	0.528	0.638
4	yolov8s_ft_3		0.723	0.699	0.556	0.642

En los resultados por clase, se observa que:

- Dent y Wreck presentan los valores más altos de recall en todos los modelos.
- Broken glass mantiene un rendimiento estable entre configuraciones.
- Scratch es la clase con menor rendimiento en todas las métricas, especialmente en mAP@0.5:0.95.

Resultados de la evaluación cualitativa



En la evaluación cualitativa, los modelos muestran una correcta detección de los daños más grandes, mientras que presentan más dificultad en la detección de daños pequeños.

	Modelo	Total	FP	FP_rate
0	YOLOv8s	100	33	0.33
1	YOLOv8n	100	54	0.54
2	YOLOv8s_ft	100	43	0.43
3	YOLOv8s_ft_2	100	31	0.31
4	YOLOv8s_ft_3	100	27	0.27

Podemos observar también diferencias en la confianza de las predicciones y en la cobertura

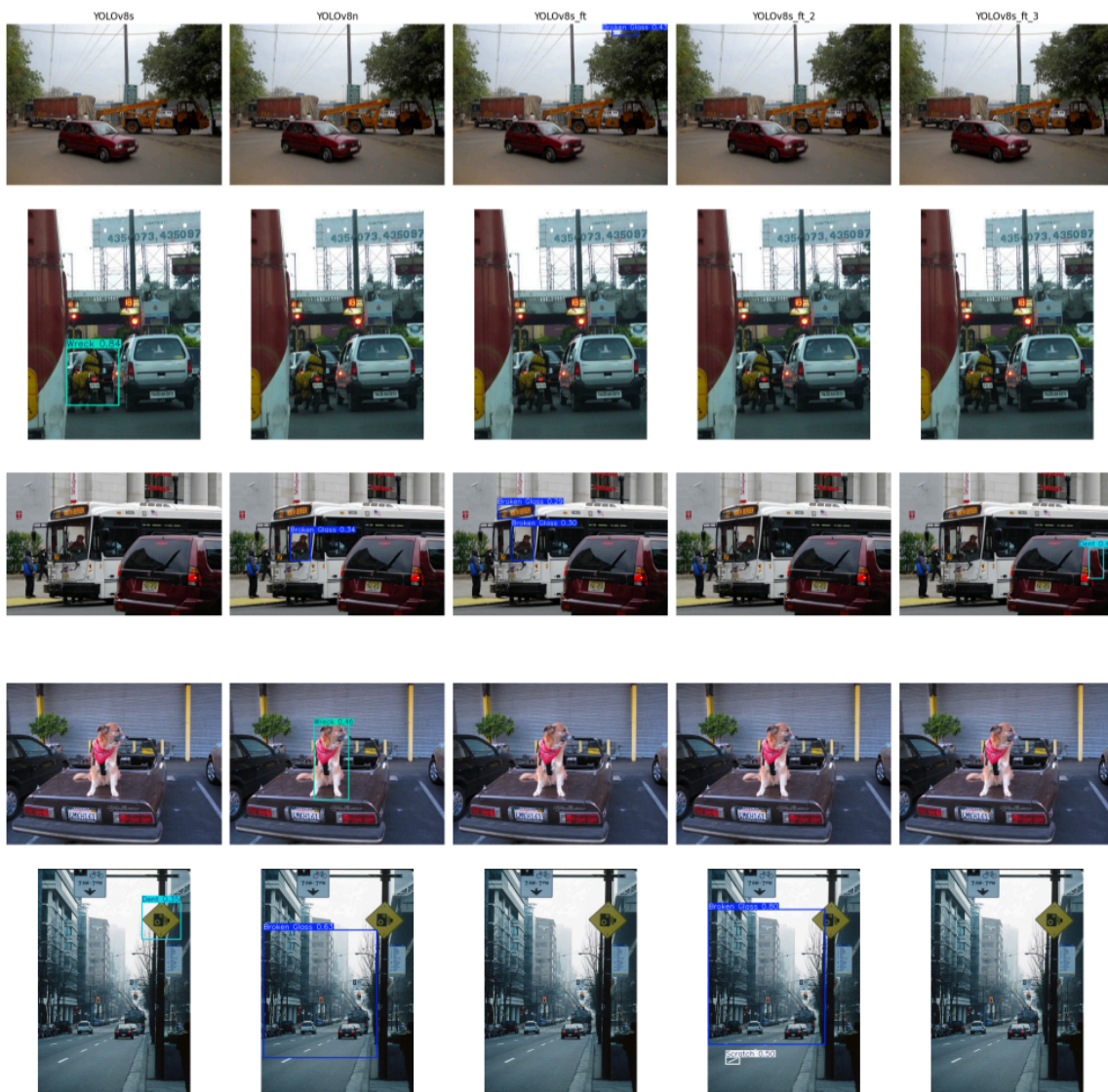
de las bounding boxes entre los distintos modelos.

	Modelo	Dent	Wreck	Broken Glass	Scratch
0	YOLOv8s	5.0	20	8	10
1	YOLOv8n	1.0	46	15	11
2	YOLOv8s_ft	0.0	17	28	9
3	YOLOv8s_ft_2	2.0	14	19	8
4	YOLOv8s_ft_3	1.0	19	7	7

Falsos positivos y distribución de falsos positivos por clase

En cuanto a los falsos positivos, el modelo baseline presenta una tasa de 0.33, mientras que los modelos ajustados muestran una reducción progresiva, alcanzando el valor más bajo en yolov8s_ft_3 con 27%. La distribución de falsos positivos varía entre clases, siendo más frecuentes en Wreck y Broken glass.

Ejemplos de falsos positivos



7. Análisis crítico

7.1 Limitaciones del enfoque y problemas encontrados

A pesar de que el modelo YOLO en sus distintas variantes mostró un rendimiento sólido en el problema de detección de daños en vehículos, el enfoque presenta varias limitaciones relevantes.

En primer lugar, identificamos una dificultad persistente en la detección de daños pequeños, en especial en la clase Scratch. Este problema se mantuvo incluso después de probar varias estrategias de data augmentation y aumentar la resolución de las imágenes de entrada, lo que sugiere que el problema no se debe únicamente a la configuración del modelo, sino también a la propia naturaleza del dataset y cómo se representan los distintos objetos a detectar. La combinación de objetos pequeños, baja relación entre señal y ruido y el preprocesado de stretch aplicado en Roboflow, pudieron degradar la representación espacial de los daños, dificultando el aprendizaje del modelo.

En segundo lugar, las anotaciones originales del dataset, basadas en polígonos de segmentación, han sido transformadas a bounding boxes para adaptarse al modelo YOLO. Aunque esta decisión simplifica el problema y mejora la eficiencia computacional, introduce una pérdida de precisión en la localización del daño. Esta simplificación puede afectar negativamente tanto al entrenamiento como a la evaluación del modelo.

Asimismo, se observó un desbalance moderado entre las clases. Aunque no se aplicaron técnicas de ponderación al considerarlo una distribución razonablemente realista, puede que haya influido en el aprendizaje del modelo, afectando la sensibilidad a ciertas clases.

Por otra parte, la incorporación del conjunto de imágenes negativas procedente de COCO ha permitido evaluar cómo se comportaba el modelo frente a elementos ambiguos (árboles, señales, etc.), identificando patrones de falsos positivos. Sin embargo, estas imágenes no se han integrado durante el entrenamiento, por lo que el modelo no ha podido aprender explícitamente a discriminar entre daño real y ruido, lo que limita su robustez en contextos reales.

Finalmente, se observa que ciertas técnicas de data augmentation, como mixup, no solo no aportan mejoras, sino que pueden degradar el rendimiento del modelo. Esto sugiere que la generación de combinaciones artificiales poco realistas puede interferir en el aprendizaje de patrones específicos del dominio, especialmente en tareas donde la localización precisa es crítica.

7.2 Posibles mejoras

Como posibles mejoras, planteamos distintas propuestas.

En primer lugar, se podrían explorar técnicas de balanceo de clases, para reducir el impacto del desbalance observado en el dataset y mejorar el rendimiento en clases minoritarias.

En segundo lugar, como observamos que los resultados se optimizaron con una mejor resolución de imágenes de entrada, se podría explorar un incremento adicional a 832 píxeles o bien utilizar técnicas de multi-scale training, entrenando el modelo con unas imágenes con resolución variada. Esto permitiría mejorar la generalización frente a objetos de distintos tamaños.

Por otra parte, se podrían incorporar técnicas adicionales de augmentation específicas, como blur, o simular condiciones meteorológicas adversas, para mejorar la robustez del modelo en escenarios reales.

Finalmente, dado el comportamiento observado con *mixup*, sería recomendable evitar su uso en este tipo de problema o, alternativamente, estudiar configuraciones más controladas de esta técnica, ya que su efecto en tareas de detección de objetos pequeños parece ser negativo en este caso concreto.

8. Conclusiones

A lo largo de este proyecto se han evaluado distintas configuraciones del modelo YOLOv8 aplicadas a la detección de daños en vehículos, combinando métricas cuantitativas, análisis cualitativo y estudio de falsos positivos. El objetivo ha sido analizar el comportamiento del modelo bajo diferentes ajustes de arquitectura, data augmentation y resolución de entrada.

El modelo baseline YOLOv8s se ha consolidado como la opción más estable, ofreciendo el mejor equilibrio entre precision y recall. Este modelo demuestra una capacidad sólida para la detección de daños claramente visibles, aunque mantiene limitaciones en la identificación de daños pequeños (arañazos).

Por otro lado, la variante YOLOv8n presenta un rendimiento inferior en todas las métricas evaluadas, acompañado de un incremento notable en falsos positivos. Esto confirma que la reducción de capacidad del modelo afecta directamente a su capacidad de generalización y discriminación en este tipo de problemas.

En cuanto a las distintas configuraciones de fine-tuning, se observa que la introducción de técnicas como mixup no mejora el rendimiento del modelo en este dominio, e incluso puede introducir patrones poco realistas que dificultan el aprendizaje de características relevantes del daño.

La mejora más relevante en este conjunto de experimentos pasa por incrementar la resolución de entrada a 768 píxeles, lo que permite capturar mejor los detalles espaciales. Aunque esta modificación no resuelve completamente la detección de daños

pequeños, sí contribuye a mejorar la estabilidad del modelo y reduce la tasa de falsos positivos, obteniendo el mejor equilibrio global entre todas las configuraciones evaluadas.

En conclusión, la configuración basada en YOLOv8s con ajuste de data augmentation (sin mixup) y aumento de resolución a 768 píxeles representa la mejor solución obtenida, al ofrecer el mejor compromiso entre rendimiento, robustez y capacidad de generalización en escenarios realistas.

Como líneas de trabajo futuras, se proponen varias direcciones:

- incrementar la resolución de entrada (por ejemplo, hasta 832 píxeles o superior), con el objetivo de mejorar la detección de daños pequeños, aunque a costa de un mayor coste computacional.
- se podría explorar el entrenamiento multi-scale, utilizando imágenes a distintas resoluciones para mejorar la robustez del modelo frente a variaciones de escala, si bien esto puede incrementar la complejidad del entrenamiento.
- finalmente, una mejora relevante sería la incorporación explícita de imágenes negativas (no damage) durante el entrenamiento, con el objetivo de reducir la tasa de falsos positivos en escenarios reales.

9. Referencias

Ayyadevara, V. K., & Reddy, Y. (2023). Modern Computer Vision with PyTorch (2nd ed.). Packt Publishing.

Lakshmanan, V., Görner, M., & Gillard, R. (2022). Practical Machine Learning for Computer Vision. O'Reilly Media.

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. <https://arxiv.org/abs/1506.02640>

Ultralytics (2023). YOLOv8 Documentation. <https://docs.ultralytics.com/>

Ultralytics GitHub Repository. YOLOv8. <https://github.com/ultralytics/ultralytics>

COCO Dataset. Microsoft Common Objects in Context. <https://cocodataset.org/>

Roboflow Universe. Car Damage V5 Dataset. <https://universe.roboflow.com/>

PyTorch Documentation. <https://pytorch.org/docs/stable/index.html>